

# CBODS — Panel Prep

---

*One page of what to say, show, and answer. Numbers and claims are verified against the code as of September 2026 (see README and docs/ for the details).*

*For the slide-by-slide version, see [PRESENTATION\_OUTLINE.md](PRESENTATION\_OUTLINE.md).*

## Elevator pitch

---

*CBODS is a Django web platform that connects blood donors, patients and hospital blood banks in one audited workflow. Donors register with government-ID verification and are steered — by live stock data — to the hospitals that need their blood group most. Patients request blood with automatic ABO/Rh compatibility matching. Hospital staff screen donors, record donations, clear each unit through laboratory TTI testing, and reserve and issue bags under concurrency-safe controls. It adds organ-donation requests, which mainstream blood systems do not handle.*

One-line positioning: **\*\*NBSG's website mobilises donors by broadcast; BSIS tracks units inside the blood service; CBODS connects donors <-> hospitals <-> patients with audited, race-safe transactions — and now models the lab gate.\*\***

## Headline claims (each with proof in the code)

---

1. **\*\*Nothing computed is stored.\*\*** Stock is counted from AVAILABLE `BloodBag` rows; a donor's next eligible date is derived from `Donation` rows. No counter exists that can drift — two pages cannot disagree.
2. **\*\*A bag can never be issued twice.\*\*** Reserve and issue run inside `transaction.atomic()` with `select\_for\_update()`, re-checking status under the lock; `BloodBag.reserved\_for` pins each reservation to one request. Covered by a dedicated race-safety test.
3. **\*\*Safety gates live in the service layer, not the UI.\*\*** Eligibility is re-checked in `record\_donation`; compatibility is a data dict enforced in service functions; every bag is born UNTESTED and `AVAILABLE` is reachable only through `record\_tti\_results` with all four markers non-reactive — the gate is structural, so there is no per-view check to forget.
4. **\*\*Every mandated action is audited, and the trail survives deletion.\*\***

`AuditLog` references entities loosely (`entity\_type` + `entity\_id`);

`set\_bag\_status` is the single choke point for bag transitions.

5. **\*\*Privacy is enforced by construction.\*\*** `MEDIA\_ROOT` has no URL route at all — ID scans stream only through an ADMIN-gated view; staff are scoped to their hospital; patients and donors never see each other's identities.

6. **\*\*Graceful degradation on real Ghanaian networks.\*\*** Swappable SMS providers (console / SasuSync / Africa's Talking), SMTP with retry and timeout, and console backends so a demo never needs a signup or signal.

## Key numbers

| Fact                               | Value                                                          |
|------------------------------------|----------------------------------------------------------------|
| Stack                              | Python / Django 5.2, server-rendered templates, 2 runtime deps |
| Models / apps / tests              | 12 / 8 / 228 (incl. a full end-to-end HTTP walkthrough)        |
| TTI markers per donation           | 4 — HIV, hepatitis B, hepatitis C, syphilis                    |
| Bag shelf life / donation interval | 35 days / 90 days (derived, not stored)                        |
| Donor rules                        | age 18–60, weight ≥ 50 kg, Hb ≥ 12.5 g/dL, BP 90–180 / 60–100  |
| Sessions / reset links             | 15-min true idle timeout / 24-hour one-time links              |
| ID upload                          | JPG, PNG or PDF, ≤ 5 MB, admin-only streaming view             |
| Alerts                             | low stock < 3 units, expiry warning 7 days                     |

## 90-second demo script

1. `demo\_donor1` / `demo12345` → **\*\*Where to donate\*\***: hospitals ranked by scarcity, book where the donor's group is short.
2. `demo\_staff1` confirms the booking; `demo\_admin` approves a pending donor (ID scan behind the gated view).
3. Staff record the donation → bag appears **\*\*UNTESTED\*\*** → **\*\*Inventory** → TTI screening → enter four non-reactive results → bag is AVAILABLE.
4. `demo\_patient1` requests blood → staff accept (FEFO reserve) → fulfil.
5. **\*\*Audit log\*\***: every step above, one row each.

## Anticipated questions

- **Why SQLite?** Dev/demo default; the ORM isolates the app, production swaps to PostgreSQL by changing one setting. The concurrency guarantees come from `select\_for\_update()` row locks, which PostgreSQL honours fully.

- **Did you build the laboratory side?** Yes — the TTI gate (four markers, reactive => discard + donor notification, partial entry refused). Confirmatory assays, component processing and inter-facility distribution are out of scope; that is BSIS's regulated domain.
- **What happens when the network fails?** Notification emails/SMS queue through retry or console backends instead of breaking the workflow. The web app itself needs connectivity — a named boundary (BSIS solves it with local deployment).
- **Data Protection Act, 2012 (Act 843)?** Design intent, not certification: purpose limitation, gated ID access, role-scoped querysets, full audit trail, 15-minute idle sessions.
- **Why a monolith, no React SPA?** An MVP needs correctness more than sparkle: Django's auth, CSRF, admin and migrations shrink the error surface, and server-rendered pages work well on low-end devices.
- **How do you know it works?** 228 tests, including double-issue race safety, per-request reservation isolation, privacy partitions, and one test that drives the entire demo story over real HTTP.
- **What would you build next?** Adverse-reaction reporting (NBSG has a form for this), inter-hospital transfers, a donor-deferral registry driven by TTI results, then PostgreSQL deployment.

## Honest known gaps (offer these before they're found)

---

Adverse-reaction reporting is not modelled · no component processing (one whole-blood bag per donation) · no inter-hospital transfer · single-country scope with hardcoded Ghana rules by design.